```
"""
verify_grus_seal.py
===================
Standalone GRUS Seal Verifier
Specification: GRUS-SEAL-001-v1.0 Section 2

Any third party may run this script to verify any GRUS-AUDIT-NNNN
certification record. No connection to the LLC required. No special
permissions required. The verification is purely mathematical.

USAGE:
    # Verify a record file using the canonical LLC public key:
    python3 verify_grus_seal.py path/to/audit_record.yaml \\
        --public-key path/to/public_key.pem

    # Or with the public key fetched from grus.utility:
    python3 verify_grus_seal.py path/to/audit_record.yaml --fetch-key

EXIT CODES:
    0 = seal verifies (record IS GRUS-certified)
    1 = seal does not verify (record is NOT GRUS-certified, or has been modified)
    2 = error (missing dependency, file not found, malformed input)

Published by: Green Recursive Utility Service LLC
License: MIT (this verifier is open source for public auditability)
"""

import sys
import os
import base64
import hashlib
import argparse
import urllib.request
import yaml

try:
    from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PublicKey
    from cryptography.hazmat.primitives import serialization
except ImportError:
    print("ERROR: cryptography library not installed.")
    print("Install: pip install cryptography pyyaml")
    sys.exit(2)


CANONICAL_PUBLIC_KEY_URL = "https://grus.utility/public-key"


def canonicalize_yaml(record: dict) -> bytes:
```

```
    record_no_seal = {k: v for k, v in record.items() if k != "grus_seal"}
    return yaml.safe_dump(
        record_no_seal, sort_keys=True, default_flow_style=False
    ).encode("utf-8")


def load_public_key_from_file(path: str):
    with open(path, "rb") as f:
        return serialization.load_pem_public_key(f.read())


def fetch_public_key(url: str = CANONICAL_PUBLIC_KEY_URL):
    req = urllib.request.Request(url, headers={"User-Agent": "GRUS-Verifier/1.0"})
    with urllib.request.urlopen(req, timeout=30) as resp:
        return serialization.load_pem_public_key(resp.read())


def verify(record_path: str, public_key_path: str = None, fetch_key: bool = False) -> int:
    if not os.path.exists(record_path):
        print(f"ERROR: record file not found: {record_path}")
        return 2

    with open(record_path, "r", encoding="utf-8") as f:
        record = yaml.safe_load(f)
    if not isinstance(record, dict):
        print("ERROR: record file does not contain a YAML mapping")
        return 2

    seal_str = record.get("grus_seal", "")
    audit_id = record.get("audit_id", "(unknown)")

    print(f"Verifying record: {audit_id}")
    print(f"  Issued by: {record.get('issued_by', '(missing)')}")

    if not seal_str:
        print("  RESULT: ❌ no grus_seal field present — record is NOT signed")
        return 1

    if seal_str.startswith("ed25519-EPHEMERAL:"):
        print("  RESULT: ⚠ EPHEMERAL seal detected")
        print("  This is a development or test run, NOT a GRUS-certified record.")
        print("  Production GRUS-certified records carry seals prefixed 'ed25519:' (no -
EPHEMERAL).")
        return 1

    if not seal_str.startswith("ed25519:"):
        print(f"  RESULT: ❌ unrecognized seal format: {seal_str[:30]}")
        return 1

    # Get public key
```

```
    if fetch_key:
        print(f"  Fetching public key from {CANONICAL_PUBLIC_KEY_URL}...")
        try:
            pub = fetch_public_key()
        except Exception as e:
            print(f"  ERROR: could not fetch public key: {e}")
            return 2
    elif public_key_path:
        if not os.path.exists(public_key_path):
            print(f"  ERROR: public key file not found: {public_key_path}")
            return 2
        pub = load_public_key_from_file(public_key_path)
    else:
        print("  ERROR: must specify --public-key <path> or --fetch-key")
        return 2

    # Verify
    seal_bytes = base64.b64decode(seal_str[len("ed25519:"):])
    canonical = canonicalize_yaml(record)
    canonical_hash = hashlib.sha256(canonical).digest()

    try:
        pub.verify(seal_bytes, canonical_hash)
        print("  RESULT: ✓ SEAL VERIFIES")
        print(f"  Record IS GRUS-certified.")
        print(f"  Audit result: {record.get('audit_result', '(unknown)')}")
        print(f"  Issued under: {record.get('issued_under_charter', '(missing)')}")
        return 0
    except Exception as e:
        print(f"  RESULT: ❌ SEAL VERIFICATION FAILED")
        print(f"  Record is NOT GRUS-certified, OR has been modified since issuance.")
        print(f"  Reason: {e}")
        return 1


def main():
    ap = argparse.ArgumentParser(
        description="Verify a GRUS certification record's ed25519 seal."
    )
    ap.add_argument("record", help="Path to certification record YAML file")
    ap.add_argument("--public-key", help="Path to GRUS public_key.pem")
    ap.add_argument("--fetch-key", action="store_true",
                    help=f"Fetch the canonical public key from {CANONICAL_PUBLIC_KEY_URL}")
    args = ap.parse_args()

    if not args.public_key and not args.fetch_key:
        print("ERROR: must specify --public-key <path> or --fetch-key")
        sys.exit(2)

    sys.exit(verify(args.record, args.public_key, args.fetch_key))
```

```
if __name__ == "__main__":
    main()
```